الوراثة(Inheritance)

1 ما هي الوراثة؟

الوراثة تعني أن **كلاساً جديداً** يرث خصائص ودوال **كلاس موجود مسبقاً**، بدلاً من كتابتها من الصفر. الكلاس القديم يُسمى **Base Class (الأساس)**، والجديد يُسمى **Derived Class (المشتق)**

الوراثة تُعبّر عن علاقة **"IS-A"** — أي أن كل كائن من الكلاس المشتق **هو أيضاً** كائن من الكلاس الأساسي. مثلاً: المدير **هو** موظف.

> ⚠️ فخ امتحاني: محدد الوصول في الوراثة افتراضيه `private` إذا لم تكتبه. أي أن `class B : A` تعني وراثة خاصة تلقائياً

2 محددات الوصول في الوراثة — جدول الجوهرة

هذا الجدول **أهم شيء في الوحدة كلها** وسؤاله يتكرر في كل امتحان

وضع العضو في الأساس	وراثة `public`	وراثة `private`	وراثة `protected`
`public`	يصبح `public` ✅	يصبح `private`	يصبح `protected`
`protected`	يصبح `protected`	يصبح `private`	يصبح `protected`
`private`	❌ لا يُورَّث	❌ لا يُورَّث	❌ لا يُورَّث

> 📌 ملخص سريع: الأعضاء الـ `private` في الكلاس الأساسي لا تُورَّث أبداً بأي طريقة. إذا أردت أن يرثها الكلاس المشتق، اجعلها `protected`

من يستطيع الوصول للـ `private` و `protected` ؟

- دالة `friend` للكلاس
- دوال الأعضاء داخل نفس الكلاس
- دوال الأعضاء في الكلاس المشتق (للـ `protected` فقط)

3 أنواع الوراثة الخمسة

النوع الأول — الوراثة الفردية (Single Inheritance)

كلاس واحد يرث من كلاس واحد فقط — أبسط الأنواع وأكثرها شيوعاً

```text
Worker  ──►  Manager
(أساس)       (مشتق)
```

```cpp
class worker                // الكلاس الأساسي
{
    int age;
    char name[10];
public:
    void get();
    void show();
};

class manager : public worker  // علنياً يرث من worker
{
    int now;                // عضو جديد خاص بـ manager
public:
    void get();
    void show();
};

void manager::get()
{
    worker::get();          // يستدعي دالة الأساس أولاً
    cout << "number of workers: ";
    cin >> now;
}
```

> 📌 نقطة مهمة: لاستدعاء دالة الأساس من المشتق نستخدم: `()BaseClass::functionName` — نقطتان رأسيتان `::`

النوع الثاني — الوراثة متعددة المستويات (Multilevel Inheritance)

سلسلة: A يرث منه B، وB يرث منه C. الكلاس الوسط يُسمى **Intermediate Base Class**

```text
Worker  ──►  Manager  ──►  CEO
(أساس)       (وسيط)        (مشتق)
```

```cpp
class worker  { /* ... */ };            // المستوى 1

class manager : public worker           // المستوى 2
{
    int now;
public:
    void get() { worker::get(); cin >> now; }
};

class ceo : public manager              // المستوى 3
{
    int nom;
public:
    void get() { manager::get(); cin >> nom; } // يستدعي المستوى 2
};
```

> ⚠️ فخ امتحاني: في الوراثة متعددة المستويات، `ceo` يرث كل ما ورثه `manager` من `worker` أيضاً. أي أن `ceo` لديه وصول لجميع المستويات

النوع الثالث — الوراثة المتعددة (Multiple Inheritance)

كلاس واحد يرث من **كلاسَيْن أو أكثر** في نفس الوقت — مثل الطفل الذي يرث من أب وأم

```text
Father    Mother
   \       /
    Daughter
```

```cpp
class daughter : public father, public mother
{
    char name[20];
    int std;
public:
    void get()
    {
        father::get();   // يستدعي get() من father
        mother::get();   // يستدعي get() من mother
        cin >> name >> std;
    }
};
```

الصيغة العامة:

```text
class Derived : visibility Base1, visibility Base2
{
    // ...
};
```

> ⚠️ فخ امتحاني: لو كان في `father` و `mother` كلاهما دالة اسمها `get()`، واستدعيت `d1.get()` — المُصرّف سيُعطي Ambiguity Error. الحل: تحديد الكلاس `father::get()` أو `mother::get()`

النوع الرابع — الوراثة الهرمية (Hierarchical Inheritance)

كلاس أساسي **واحد** يرث منه **عدة كلاسات** مختلفة — مثل شجرة العائلة

```text
        Father
       /       \
     Son      Daughter
```

```cpp
class father { /* أب مشترك */ };

class son : public father        // الابن يرث من الأب
{
    char name[20];
    int age;
    // ...
};

class daughter : public father  // البنت أيضاً ترث من نفس الأب
{
    char name[15];
    int age;
    // ...
};
```

> 📌 الفرق عن Multiple Inheritance: في الهرمية الأساسي واحد والمشتقات كثيرة، وفي المتعددة المشتق واحد والأساسات كثيرة

النوع الخامس — الوراثة الهجينة (Hybrid Inheritance)

مزيج من نوعَيْن أو أكثر من أنواع الوراثة في نفس البرنامج

```text
       Student
          |
        Test        Sports
           \         /
             Result
```

في هذا المثال: `Test` يرث من `Student` (وراثة فردية)، ثم `Result` يرث من `Test` و `Sports` (وراثة متعددة) — هذا هجين

```cpp
class student { protected: int r_no; /* ... */ };

class test : public student          // وراثة فردية
{ protected: int part1, part2; /* ... */ };

class sports                         // كلاس مستقل
{ protected: int score; /* ... */ };

class result : public test, public sports   // وراثة متعددة
{
    int total;
public:
    void display()
    {
        total = part1 + part2 + score;  // يصل لأعضاء كلا الأساسَيْن
        put_n(); put_m(); put_s();
        cout << "Total: " << total;
    }
};
```

4 جدول مقارنة الأنواع الخمسة — للمراجعة السريعة

النوع	الشكل	مثال حقيقي	العلاقة
Single	`A → B`	موظف → مدير	كلاس واحد يرث من واحد
Multilevel	`A → B → C`	موظف → مدير → CEO	سلسلة من الوراثة
Multiple	`A + B → C`	أب + أم → طفل	كلاس يرث من أكثر من كلاس
Hierarchical	`A → B` و `A → C`	أب → ابن و أب → بنت	أساس واحد، مشتقات عديدة
Hybrid	مزيج	طالب → اختبار + رياضة → نتيجة	تجميع نوعين أو أكثر

5 استدعاء دوال الأساس من المشتق

```cpp
class manager : public worker
{
public:
    void get()
    {
        worker::get();      // ✅ استدعاء دالة الأساس بـ ::
        cin >> now;         // تم إضافة بيانات المشتق
    }

    void show()
    {
        worker::show();     // ✅ طباعة بيانات الأساس أولاً
        cout << now;        // تم طباعة بيانات المشتق
    }
};
```

> 📌 القاعدة: دائماً استدعي دالة الأساس أولاً في دالة المشتق — هذا النمط الأكثر منطقية والأكثر ظهوراً في الأكواد

6 تمارين Tracing

تمرين (1) — ما المخرج؟

```cpp
class worker
{
    int age; char name[10];
public:
    void get()   { name="Ali"; age=27; }
    void show()  { cout << name << " " << age << endl; }
};

class manager : public worker
{
    int now;
public:
    void get()  { worker::get(); now=30; }
    void show() { worker::show(); cout << now << endl; }
};

int main()
{
    manager M1;
    M1.get();
    M1.show();
}
```

السطر	المخرج
`worker::show()` داخل `manager::show()`	`Ali 27`
`cout << now`	`30`
المخرج الكامل	`Ali 27` ثم `30`

تمرين (2) — أيٌّ منها يُعطي خطأ؟

```cpp
class A
{
private:   int x;
protected: int y;
public:    int z;
};

class B : public A
{
public:
    void test()
    {
        x = 1;   // سطر 1
        y = 2;   // سطر 2
        z = 3;   // سطر 3
    }
};
```

السطر	النتيجة	السبب
`x = 1`	❌ Compile Error	`x` خاص — لا يُورَّث
`y = 2`	✅ صحيح	`y` محمي — يُورَّث للمشتق
`z = 3`	✅ صحيح	`z` عام — يُورَّث ويبقى عاماً